Posts and Telecommunications Institute of Technology



Object-Oriented Programming (OOP)
Chapter 2

Faculty: Information Technology 1

Department: Computer Science

Instructor: Pham Hoang Viet

Email: vietph@ptit.edu.vn **Phone:** 0945296388

Chapter 2

Classes and Objects - Encapsulation

COURSE MAP

Learning outcomes

- Programming Paradigms & OOP basic concepts
- How to declare/use a class
  - Constructors, Current object – this, Member functions, Passing Arguments a Constructor/Method, Creating Objects
- Access modifiers (a way to hide some members in a class)
  - Cotrolling access to members of a class using modifiers
- Overriding methods in sub-classes
- Nested classes (Inner class, Outer class)
- Some useful classes
- Wrapper classes

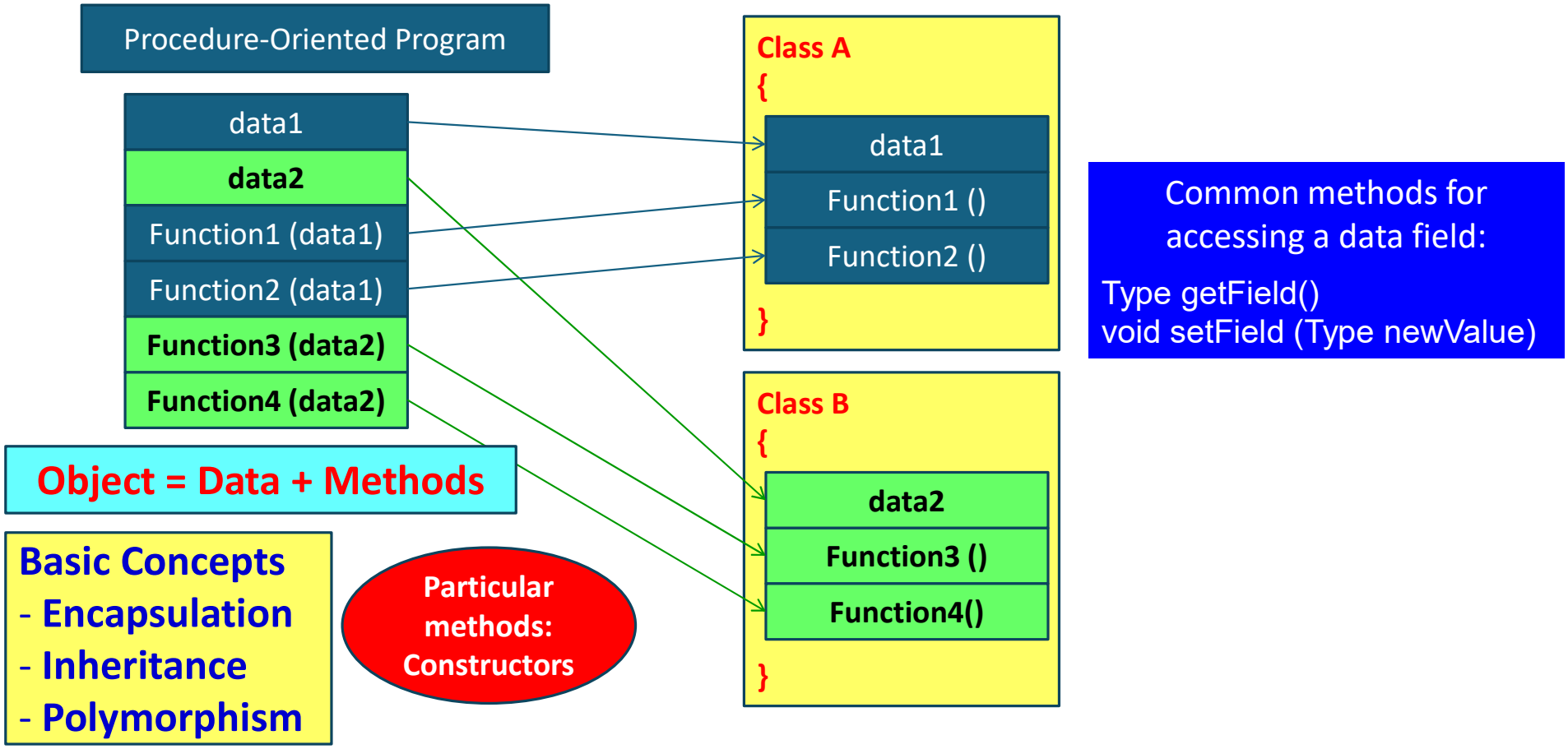
2.1. Programming Paradigms & OOP basic concepts

Programming Paradigms

High-level programming languages are divided into:

Paradigm	Description
Procedural-oriented (imperative) paradigm (POP) (3rd generation language)	Program= data + algorithms. Each algorithm is implemented as a function (group of statements) and data are it's parameters (C-language)
Object-oriented paradigm (OOP) (3rd generation language)	Programs = actions of some objects. Object = data + behaviors. Each behavior is implemented as a method (C++, Java, C#,…)
Functional paradigm (4th generation language)	Domain-specific languages. Basic functions were implemented. Programs = a set of functions (SQL)
Declarative/Logic paradigm (5th generation language)	Program = declarations + inference rules (Prolog, CLISP, …)

POP vs. OOP


Procedure-Oriented Program
data1
data2
Function1 (data1)
Function2 (data1)
Function3 (data2)
Function4 (data2)
Object = Data + Methods
Basic Concepts
- Encapsulation
- Inheritance
- Polymorphism
Particular methods: Constructors
Class A
{
data1
Function1 ()
Function2 ()
}
Class B
{
data2
Function3 ()
Function4()
}
Common methods for accessing a data field:
Type getField()
void setField (Type newValue)

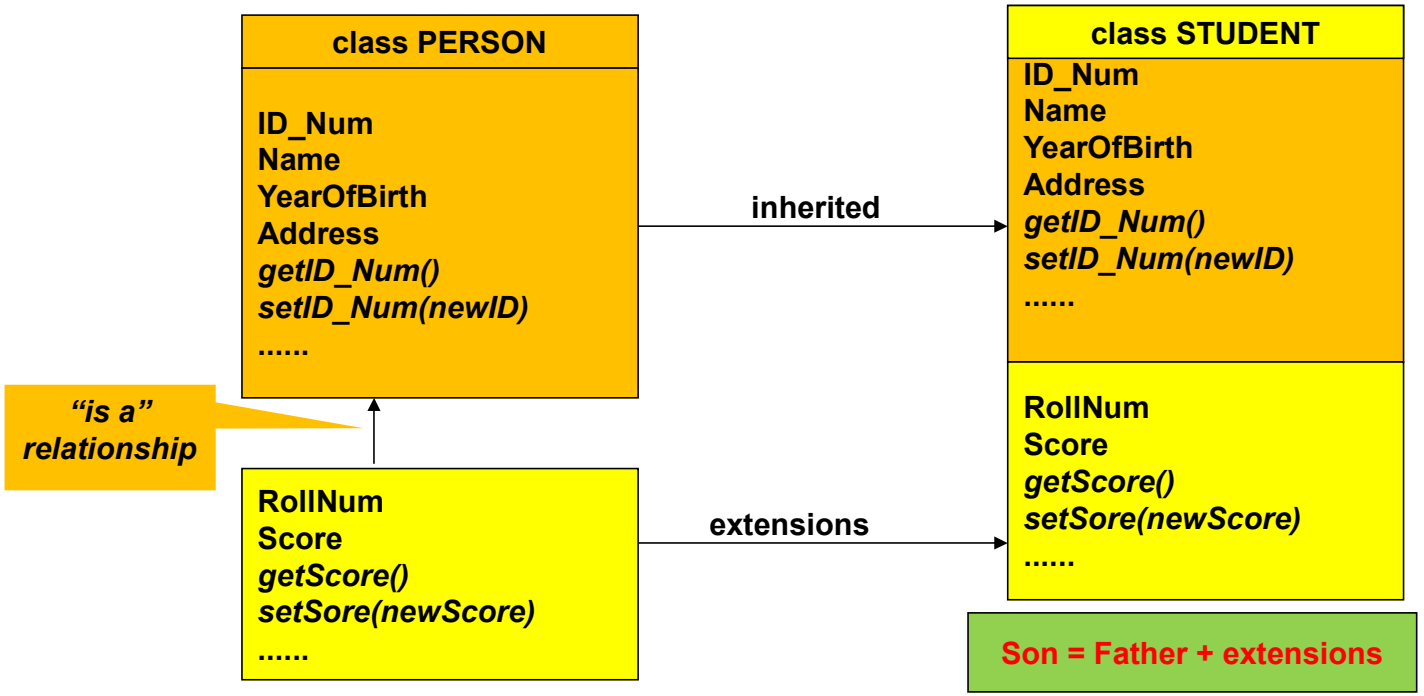
Encapsulation

Aggregation of data and behavior.

- Class = Data (fields/properties) + Methods
  - Data of a class should be hidden from the outside.
  - All behaviors should be accessed only via methods.
- A method should have a boundary condition: Parameters must be checked (use if statement) in order to assure that data of an object are always valid.
  - Constructor: A special method it's code will execute when an object of this class is initialized.
  - Getters/Setters: implementing getter and setter is one of the ways to enforce encapsulation in the program's code.

Inheritance

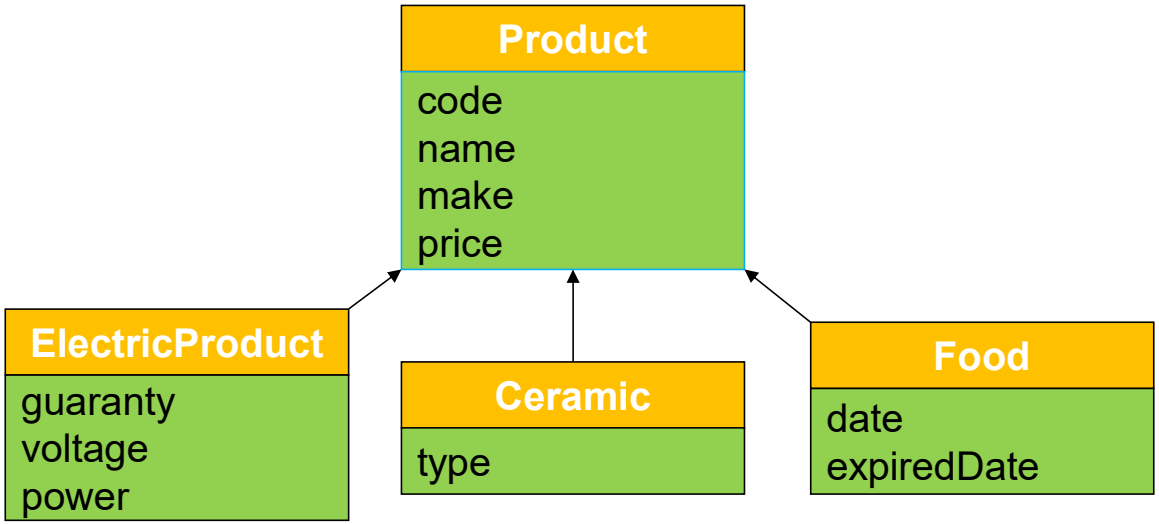
Ability allows a class having members of an existed class → Re-used code, save time



How to detect father class?

Finding the intersection of concerned classes.

- Electric Products< code, name, make, price, guaranty, voltage, power>
- Ceramic Products < code, name, make, price, type >
- Food Products < code, name, make, price , date, expiredDate >



Polymorphism

- ❖Ability allows many versions of a method based on overloading and overriding methods techniques.
- ❖Overloading: A class can have some methods which have the same name but their parameter types are different.
- ❖Overriding: A method in father class can be overridden in it’s derived classes (body of a method can be replaced in derived classes).

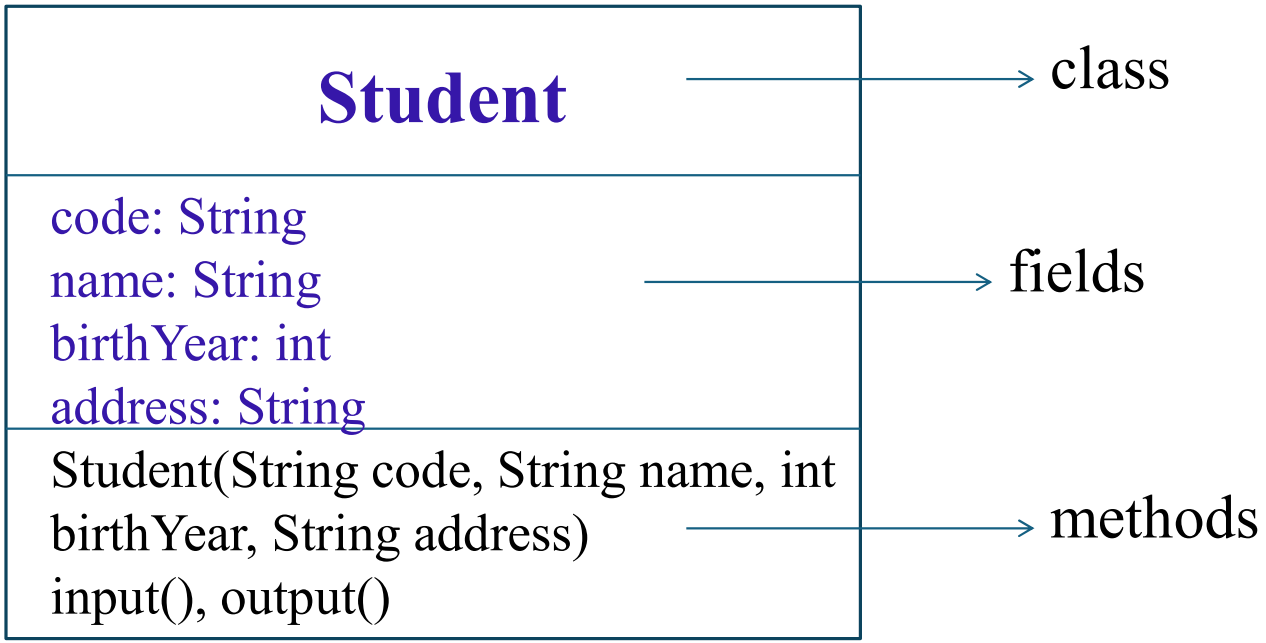
How to Identity a Class

- Main noun: Class
- Nouns as modifiers of main noun: Fields
- Verbs related to main noun: Methods

For example, details of a **Student** include **code, name, year of birth, address.**
Write a Java program that will allow **input** a student, **output** his/her.

Main noun: Student
Auxiliary nouns:code , name, birthYear, address;
verbs: input() , output()

A UML class diagram is used to represent the Student class



- Note: We can add more some functions

2.2. Declaring/Using a Java Class

Data types

❖**data type**: A category of data values.
- Example: integer, real number, string

❖Java data types are divided into two sets:
- **primitive types**: Java's 8 built-in *simple* data types for numbers, text characters, and logic.
  - boolean, char, byte, short, int, long, float, double
- **object types**: All other types!
  - e.g., Scanner, System, String, Math

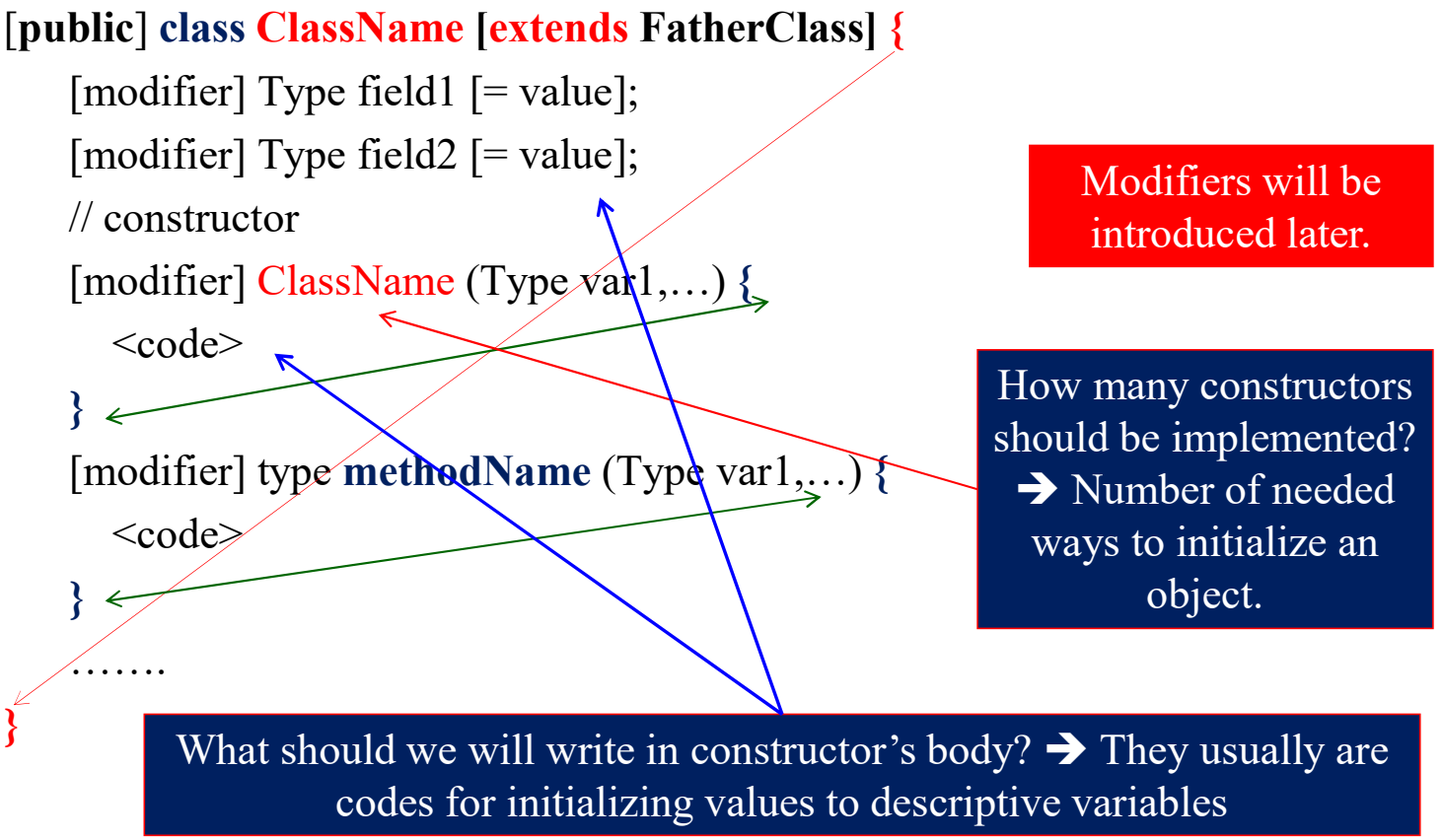
Object types

❖So far, we have seen:
- **variables**, which represent data (categorized by **types**)
- **methods**, which represent behavior

❖**object:** An variable that contains *data **and** behavior*.
- There are variables inside the object, representing its data.
- There are methods inside the object, representing its behavior.

❖**class:**
- Basic building block of Java programs (what we have seen so far)
  AND
- Data types for objects

Declaring a Java Class



❑ Example

- Theoretical examples:
  - A class Person could represent objects that store a name, height, weight, hair color, IQ, etc…
  - A class Laptop could represent objects that store speed, screen size, color, dimensions, brand, etc…

- Examples from Java:
  - The class String represents objects that store text characters.
  - The class Scanner represents objects that can *tokenize* streams of characters.

Constructors

- Constructors that are invoked to create objects from the class blueprint.
- Constructor declarations look like method declarations—except that they use the name of the class and have no return type.
- The compiler automatically provides a no-argument, default constructor for any class without constructors.
- If we implement **a** constructor, compiler does **not** insert default constructor.

```
//default constructor 1 or 2
public Student(){}   //1
public Student(){      //2
    code="B19DCCN123";
    name="To An An";
    birthYear= 2000;
    address="1 Ba Trieu , HN".
}
//constructor with parameters
public Student(String code, String name, int
birthYear, String address){
    this.code=code;
    this.name=name;
    this.birthYear= year;
    this.address=address.
}
```

The current object: this

- The keyword this returns the address of the current object.
- This holds the address of the region of memory that contains all of the data stored in the instance variables of current object.
- Scope of this: this is created and used just when the member method is called. After the member method terminates this will be discarded

❖**shadowed variable**: A field that is "covered up" by a local variable or parameter with the same name.

- Normally, it is illegal to have two variables in the same scope with the same name, but in this case (fields and local variables) it is allowed. Why?

```
11  public class Point3D {
12      private int x;
13      private int y;
14      private int z;
15      public Point3D(int x, int y, int z){
            x = x;
            y = y;
            z = z;
19      }
```

```
11  public class Point3D {
12      private int x;
13      private int y;
14      private int z;
15      public Point3D(int x, int y, int z){
16          this.x = x;
17          this.y = y;
18          this.z = z;
19      }
```

- Otherwise, to avoid shadowing, we would always have to give the parameters different names:

```
public Point3D(int initialX, int initialY, int initialZ) {
    x = initialX;
    y = initialY;
    z = initialZ;
}
```

Member functions

- Member functions are the functions, which have their declaration inside the class definition and work on the data members of the class.
- Typical method declaration:

```
[modifier] ReturnType methodName (params) {
    <code>
}
```

- Signature: data help identifying something
- Method Signature: name + order of parameter types

Getter/Setter

- A getter is a method that gets the value of a property.
- A setter is a method that sets the value of a property.
- Uses:
  - for completeness of encapsulation
  - to maintain a consistent interface in case internal details change
  - Example:

```
public String getName(){
      return name;
}
public void setName(String name){
      if(! name.isEmpty())
            this.name=name;
}
```

Other methods

For example:

```
public void input(){
    //code here
}
public void output(){
    //code here
}
```

Creating Objects

- Class provides the blueprint for objects; you create an object from a class.
- Student st = new Student("B19DCCN123","To An An",2000,"1 Ba Trieu");
- Statement has three parts:
  - **Declaration**: are all variable declarations that associate a variable name with an object type.
  - **Instantiation**: The new keyword is a Java operator that creates the object (memory is allocated).
  - **Initialization**: The new operator is followed by a call to a constructor, which initializes the new object (values are assigned to fields).

Create/Use an object of a class

- Default constructor: Constructor with no parameter.
- Parametric constructor: Constructor with at least one parameter.
- Create an object

  ClassName obj1 = new ClassName();

  ClassName obj2 = new ClassName(params);

  E.g., `Student p = new Student("B19DCCN123","To An An",2000,"1 Ba Trieu");`

  `Scanner console = new Scanner(System.in);`
- Accessing a field of the object

  object.field
- Calling a method of an object

  object.method(params)

  "." is the dereferencing operator

❖We have seen three ways to call methods:

Type:	No '.' used	<class>.<method>	<variable>.<method>
When it's used:	For methods defined in the same class as they're called	For static methods defined in another class	For non-static or instance methods defined in another class
Examples:	myMethod();	Math.max(a,b)	myString.length()
		Integer.parseInt("6")	console.nextInt()
		Server.gcd(15,12)	myPoint.translate(2,2)

❖ null objects

- null is a value for all object types that says,
  "this object is a reference to nothing at all"
- We draw null objects with a slash
- Note that null objects

  have no memory reserved for their data!

```
Student p1 = null;
p1.name = "Peter"; // ERROR
```

- If you try to dereference a null object, it will cause a NullPointerException.

- If we implement a constructor, compiler does not insert system constructor

```
package point1;
public class IntPoint2 {
  int x;
  int y;

    public IntPoint2(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public int getX() {...3 lines }
    public void setX(int x) {...3 li
    public int getY() {...3 lines }
    public void setY(int y) {...3 li
}
```

```
1   package point1;
2   public class IntPoint2_Use {
3   public static void main (String[] args){
4         // Create a point using default constructor
5         // Error:Constructor InPoint2 in class IntPoint2 can
6         // not be appied to given type;required: int, int
          IntPoint2 p = new IntPoint2();
8     }
9   }
```

Quiz?

```java
package point1;
public class IntPoint2 {
    int x=7;
    int y=3;
    public IntPoint2(){
        output();
        x=100;
        y=1000;
        output();
    }
    public IntPoint2(int x, int y) {
        output();
        this.x = x;
        this.y = y;
        output();
    }
    public void output(){
        String S= "[" + x + "," + y + "]";
        System.out.println(S);
    }
}
```

```java
package point1;
public class IntPoint2_Use {
  public static void main (String[] args){
        System.out.println("Use default constructor:");
        IntPoint2 p1= new IntPoint2();
        System.out.println("Use parametric constructor:");
        IntPoint2 p2 = new IntPoint2(-7,90);
    }
}
```

Case study: Point object

- Data stored in each Point object:

Field name	Description
`x`	the point's x-coordinate
`y`	the point's y-coordinate

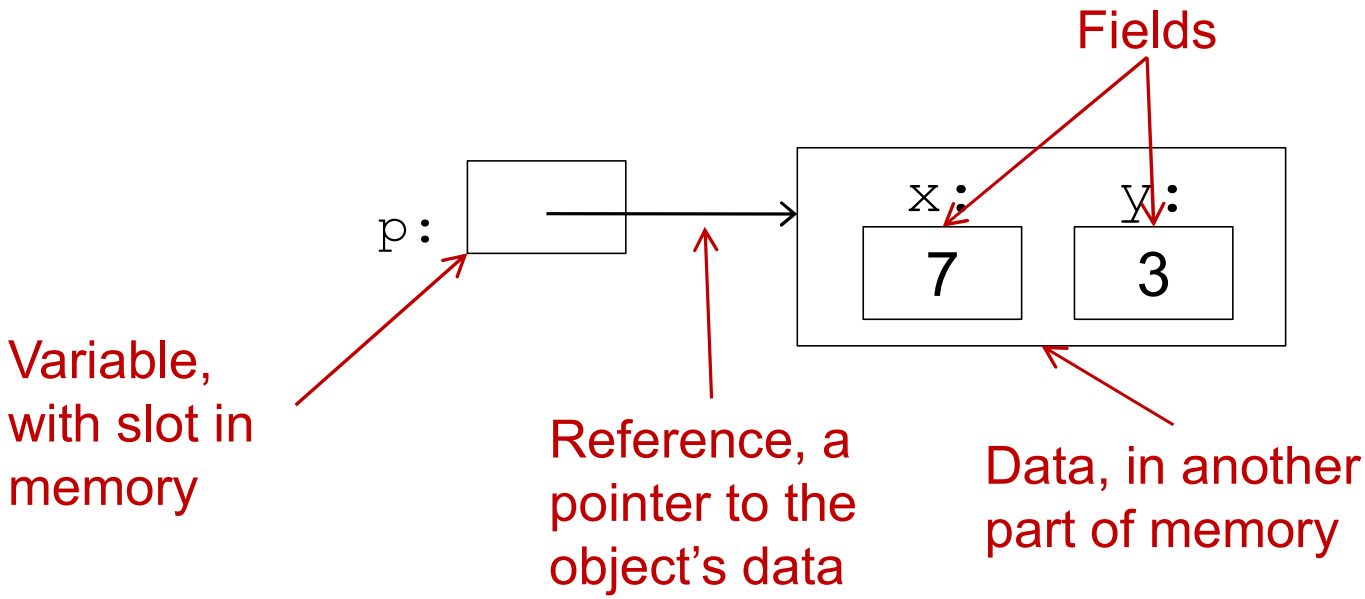
- Useful methods in each Point object:

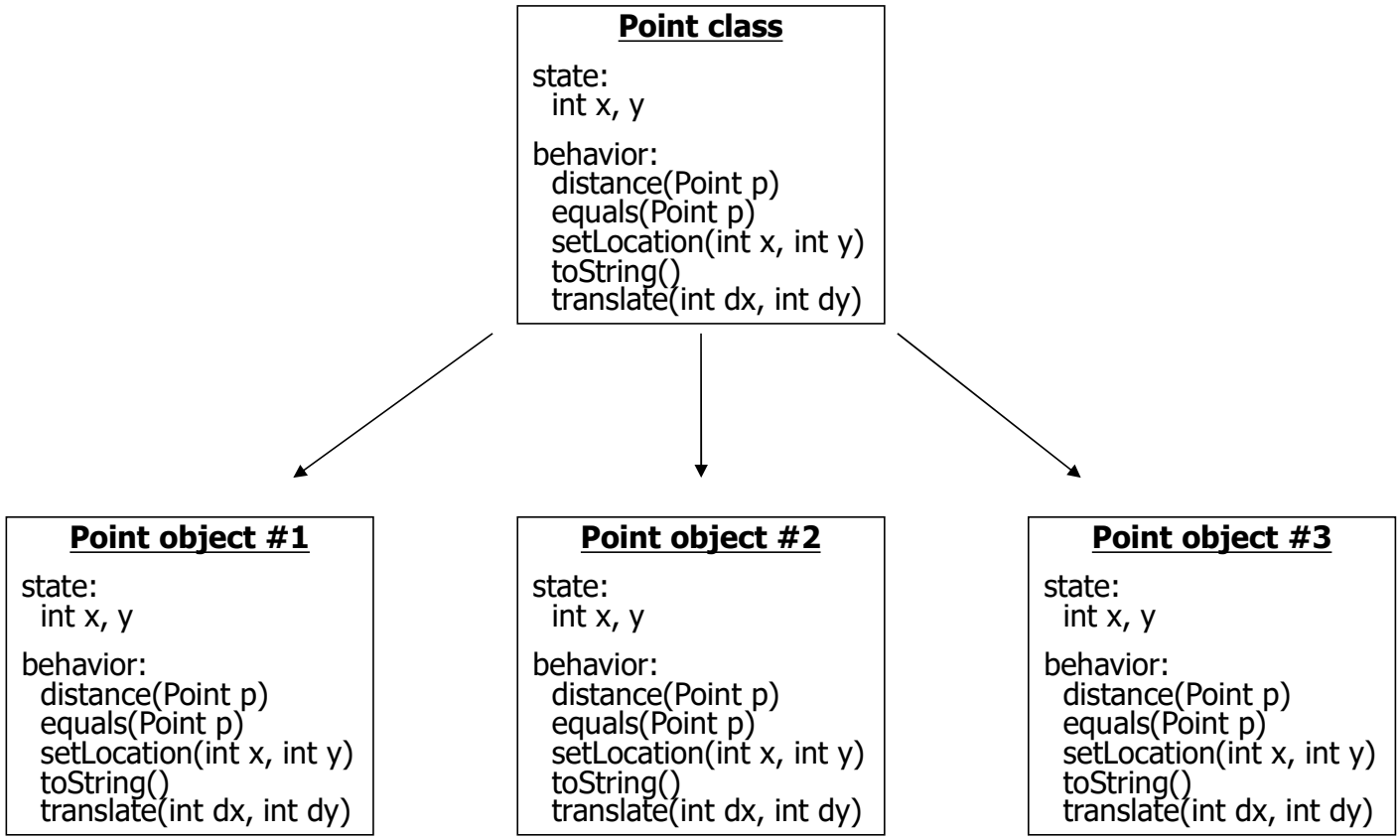
Method name	Description
`distance(`*p*`)`	how far away the point is from point *p*
`setLocation(`*x*`, `*y*`)`	sets the point's x and y to the given values
`translate(`*dx*`, `*dy*`)`	adjusts the point's x and y by the given amounts

- Constructing a Point object, general syntax:

```
Point p2 = new Point(); // the origin, (0, 0)

Point p1 = new Point(7, 3);
```






```
Point p0 = new Point(0, 0);
Point p1 = new Point(7, 3);
Point p2 = new Point(2, -2);
System.out.println(p1.distance(p0));  // 7.62
System.out.println(p2.distance(p0));  // 2.83
```

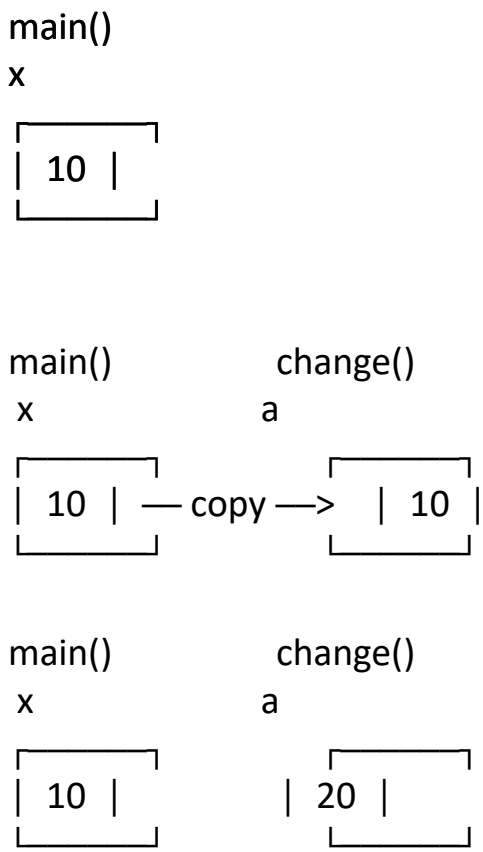
Passing Arguments a Constructor/Method

- Java uses the mechanism passing by value.
- Arguments can be:
  - Primitive Data Type Arguments:
    - Values are data
  - Reference Data Type Arguments (objects): copies of references
    - Value are references to objects.

```
static void change(int a) {
   a = 20;
}

public static void main(String[] args) {
   int x = 10;
   change(x);
   System.out.println(x);
}
```

Output: 10



```
class Student {
   String name;
}

static void changeName(Student st) {
   st.name = "John";
}

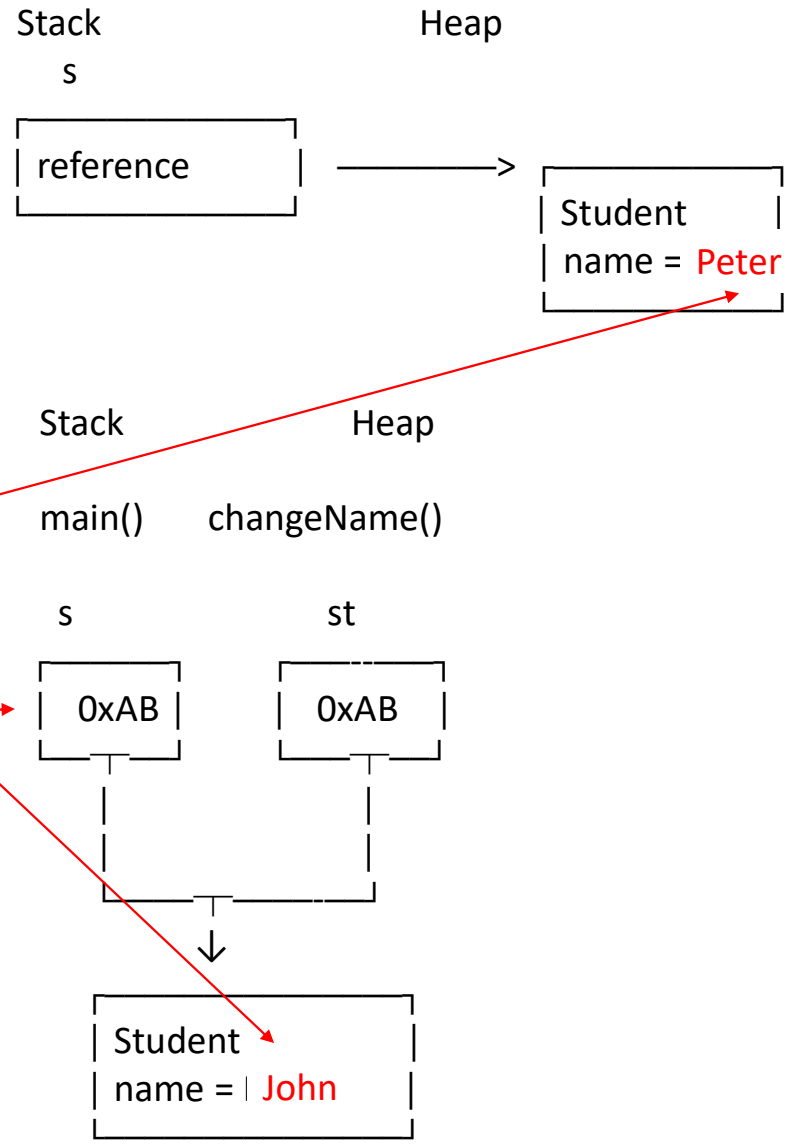
public static void main(String[] args) {

   Student s = new Student();
   s.name = "Peter";

   changeName(s);

   System.out.println(s.name);
}
```

Output: John



Explain More: Alias of Objects

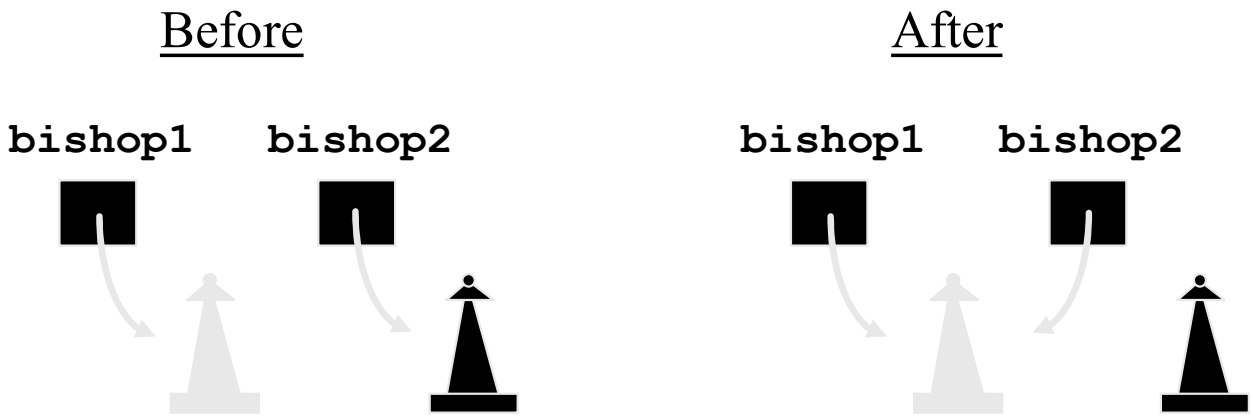
- Two or more reference variables refer to the same object.
- Example: assume a class ChessPiece

```
ChessPiece bishop1 = new ChessPiece();
ChessPiece bishop2 = new ChessPiece();
```



Assignment of Reference Types

```
bishop2 = bishop1;
```



Assignment of Primitive Types

```
num2 = num1;
```



Case study: Num.java

```
public class Num {
   private int value;

   public Num(int update) {
      value = update;
   }
   public void setValue(int update) {
      value = update;
   }
   public String toString() {
      return value + "";
   }
}
```

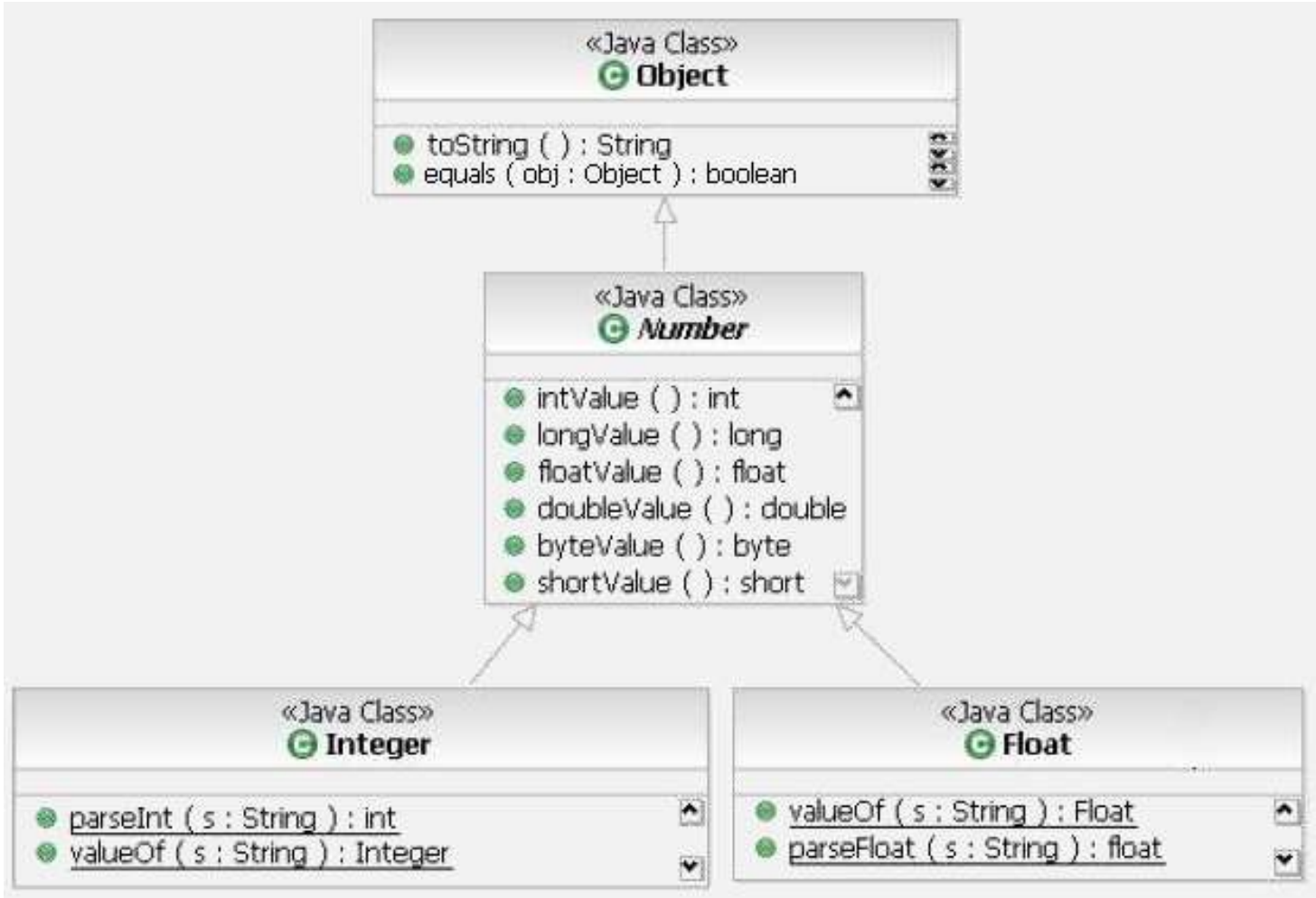
```
public class ParameterTester {
  public void changeValues (int f1, Num f2, Num f3) {
    System.out.println("Before changing the values:");
    System.out.println("f1\tf2\tf3");
    System.out.println(f1 + "\t" + f2 + "\t" +
                       f3 + "\n");
    f1 = 999;
    f2.setValue(888);
    f3 = new Num (777);
    System.out.println("After changing the values:");
    System.out.println("f1\tf2\tf3");
    System.out.println(f1 + "\t" + f2 + "\t" +
                       f3 + "\n");
  }
}
```

```
public class ParameterPassing {
  public static void main (String[] args) {
      ParameterTester tester = new ParameterTester();
      int a1 = 111;
      Num a2 = new Num (222);
      Num a3 = new Num (333);
      System.out.println("Before calling changeValues:");
      System.out.println("a1\ta2\ta3");
      System.out.println(a1 + "\t" + a2 + "\t" +
                         a3 + "\n");
      tester.changeValues(a1, a2, a3);
      System.out.println("After calling changeValues:");
      System.out.println("a1\ta2\ta3");
      System.out.println(a1 + "\t" + a2 + "\t" +
                         a3 + "\n");
   }
}
```

Output?

Object class

- The java.lang.Object class is the root of the class hierarchy. Every class has Object as a superclass. All objects, including arrays, implement the methods of this class.
- Methods
  - Class constructors: Object() This is the Single Constructor.
  - String toString() This method returns a string representation of the object.
  - boolean equals(Object obj) This method indicates whether some other object is "equal to" this one.
  - int hashCode() This method returns a hash code value for the object.
  - Class getClass() This method returns the runtime class of this Object.


«Java Class»
Object
toString ( ) : String
equals (obj : Object) : boolean
«Java Class»
Number
intValue ( ) : int
longValue ( ) : long
floatValue ( ) : float
doubleValue ( ) : double
byteValue ( ) : byte
shortValue ( ) : short
«Java Class»
Integer
parseInt (s : String) : int
valueOf (s : String) : Integer
«Java Class»
Float
valueOf (s : String) : Float
parseFloat (s : String) : float

The toString() method

- tells Java how to convert an object into a String
- called when an object is printed or concatenated to a String:

```
Point p = new Point(7, 2);
System.out.println(”p: " + p);
```

Same as:

```
System.out.println("p: " + p.toString());
```

- Every class has a toString(), even if it isn't in your code.

The default is the class's name and a hex (base-16) hash-code:

```
Point@9e8c34
```

```
public String toString() {
     //code that returns a suitable String;
}
```

- Example: toString() method for our Student class:

```
public String toString(){
  return ”name: " + name+ "\n"
    + ”id: " + id + "\n"
    + ”average: " + average;
}
```

Primitive Equality

- Suppose we have two integers i and j
- How does the statement i==j behave?
  - i==j if i and j contain the same value

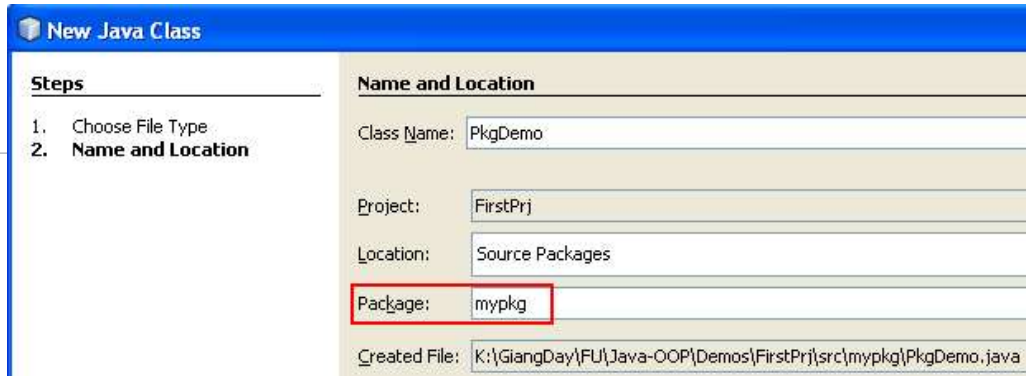
Object Equality

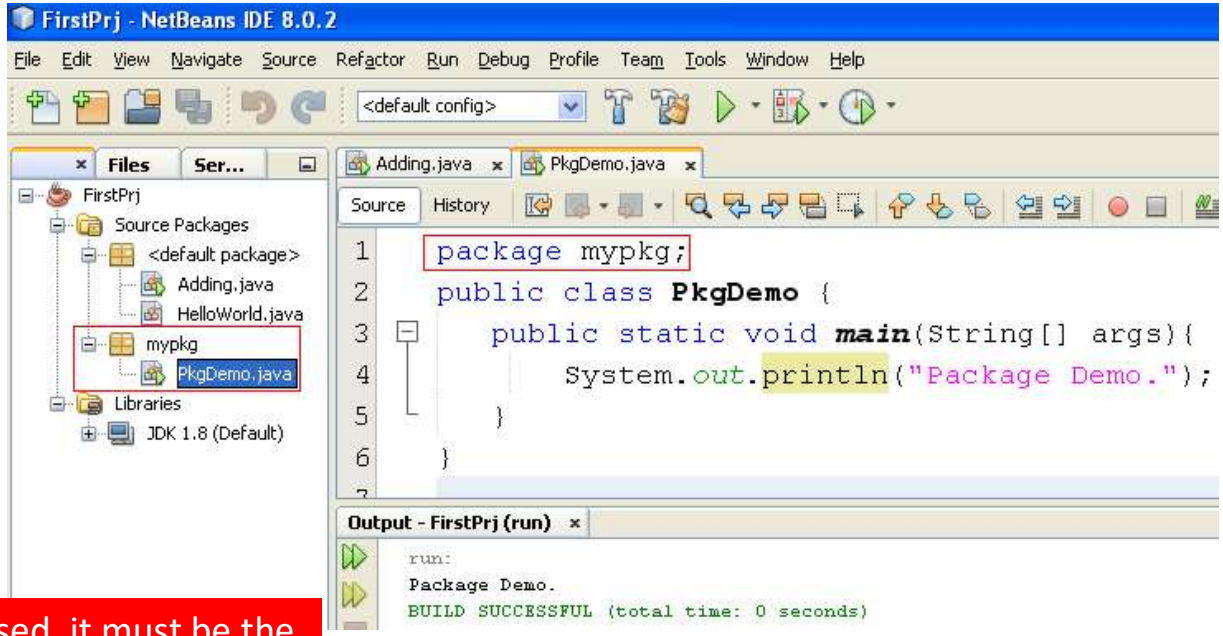
- Suppose we have two pet instances pet1 and pet2
- How does the statement pet1==pet2 behave?
  - pet1==pet2 is true if ***both*** refer to the ***same*** object
  - The == operator checks if the ***addresses*** of the two objects are equal (May not be what we want!)
- define your own **.equals()** method.
  - Use pet1.equals(pet2) instead of pet1==pet2
  - The default definition of .equals() is the value of '==', but for Strings the contents are compared

```
public boolean equals (Object other) {
  if (!other instanceof Pet) {
       return false;
   }
   Pet otherPet = (Pet) other;
  return ((this.age == otherPet.age)
     &&(Math.abs(this.weight - otherPet.weight) < 1e-8)
     &&(this.name.equals(otherPet.name)));
}
```

Package

- A package is a namespace that organizes a set of related classes and interfaces.
- The Java platform provides an enormous class library (a set of packages) suitable for use in your own applications called API.
- For example, a String object contains state and behavior for character strings.





If package is used, it must be the first line in Java code

A **package** is a set of related classes

- Syntax to put a class into a package:

```
package <your.package.name>;
public class <ClassName> { …}
```

- Two rules:
  - A package declaration must always come first in your .java file
  - Naming convention: all lower case, with '.' separating nested package names, biggest package first.

- Example:

```
package cis1068.au10.graphics;
public class Point3D { … }
```

Two options for using classes in a package:

1) Use full class name:

`<package.name>.<ClassName>`

For example,

```
java.util.Scanner console = new java.util.Scanner(System.in);
```

2) Import the package, use the "qualified" name:

At the top of the file, below any package declaration:

`import <package.name>.<ClassName>;` or

```
import <package.name>.*;
```

Then you can use `<ClassName>` anywhere, without the full package name.

For example,

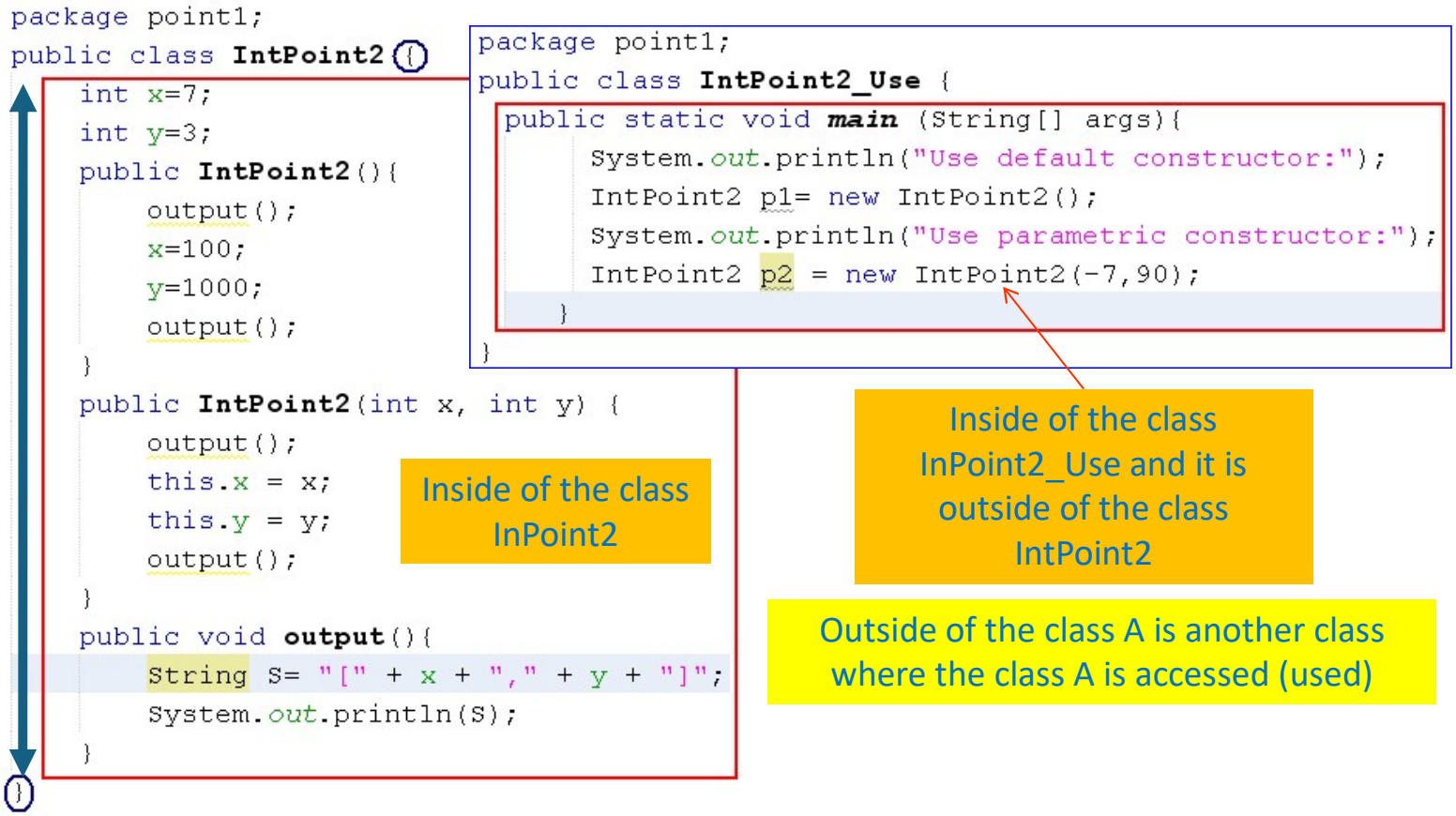
```
import java.util.Scanner; …
```
(and later) …
```
Scanner console = new Scanner(System.in);
```

2.3. Access modifiers

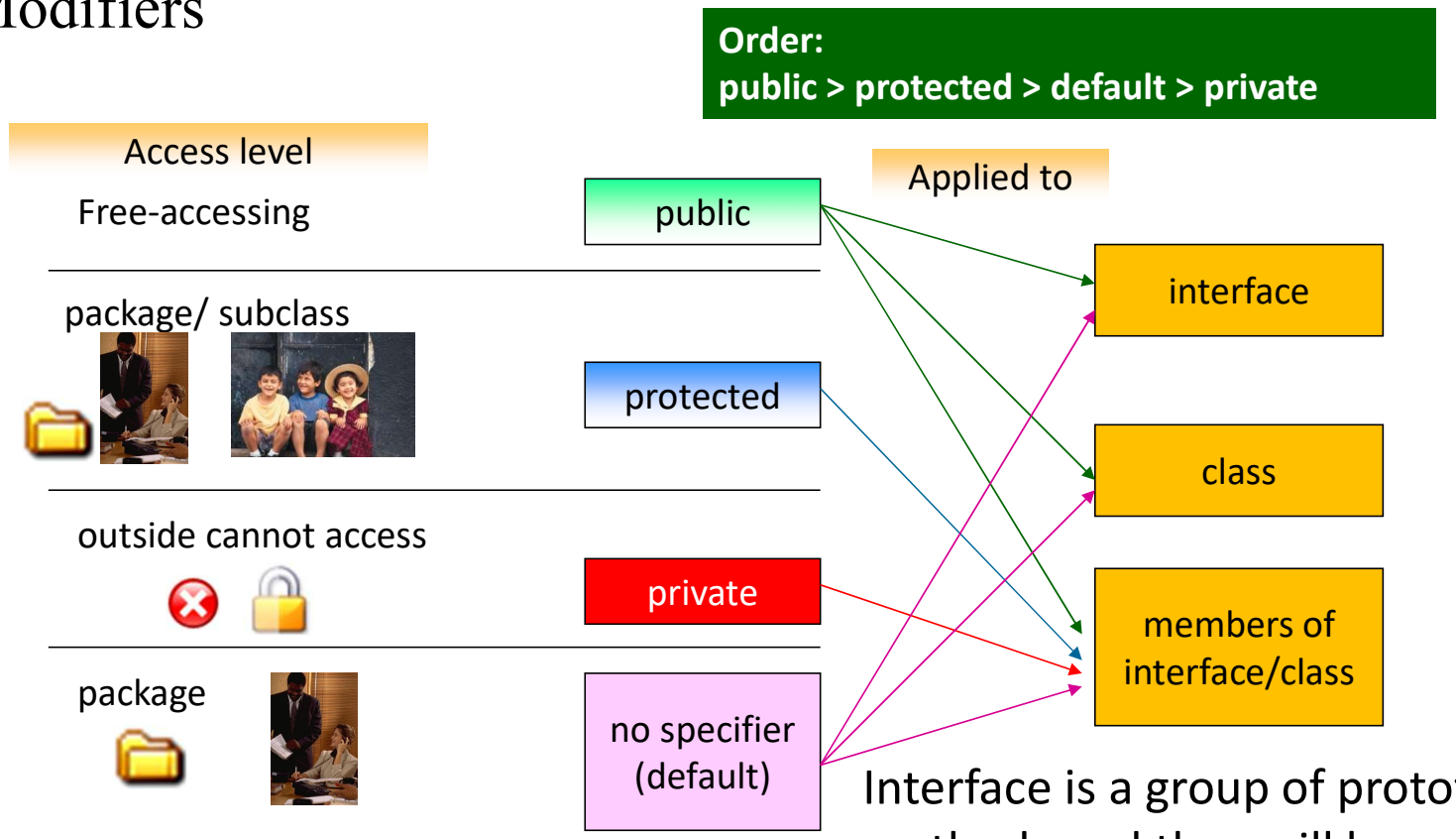
Access modifiers

- Modifier (linguistics)is a word which can bring out the meaning of other word (adjective for noun, adverb for verb)
- Modifiers (OOP) are keywords that give the compiler information about the nature of code (methods), data, classes.
  - Java supports some modifiers in which some of them are common and they are called as access modifiers (public, protected, private).
  - Access modifiers will impose level of accessing on
    - class (where it can be used?)
    - methods (whether they can be called or not)
    - fields (whether they may be read/written or not)

Outside of a Class



Access Modifiers



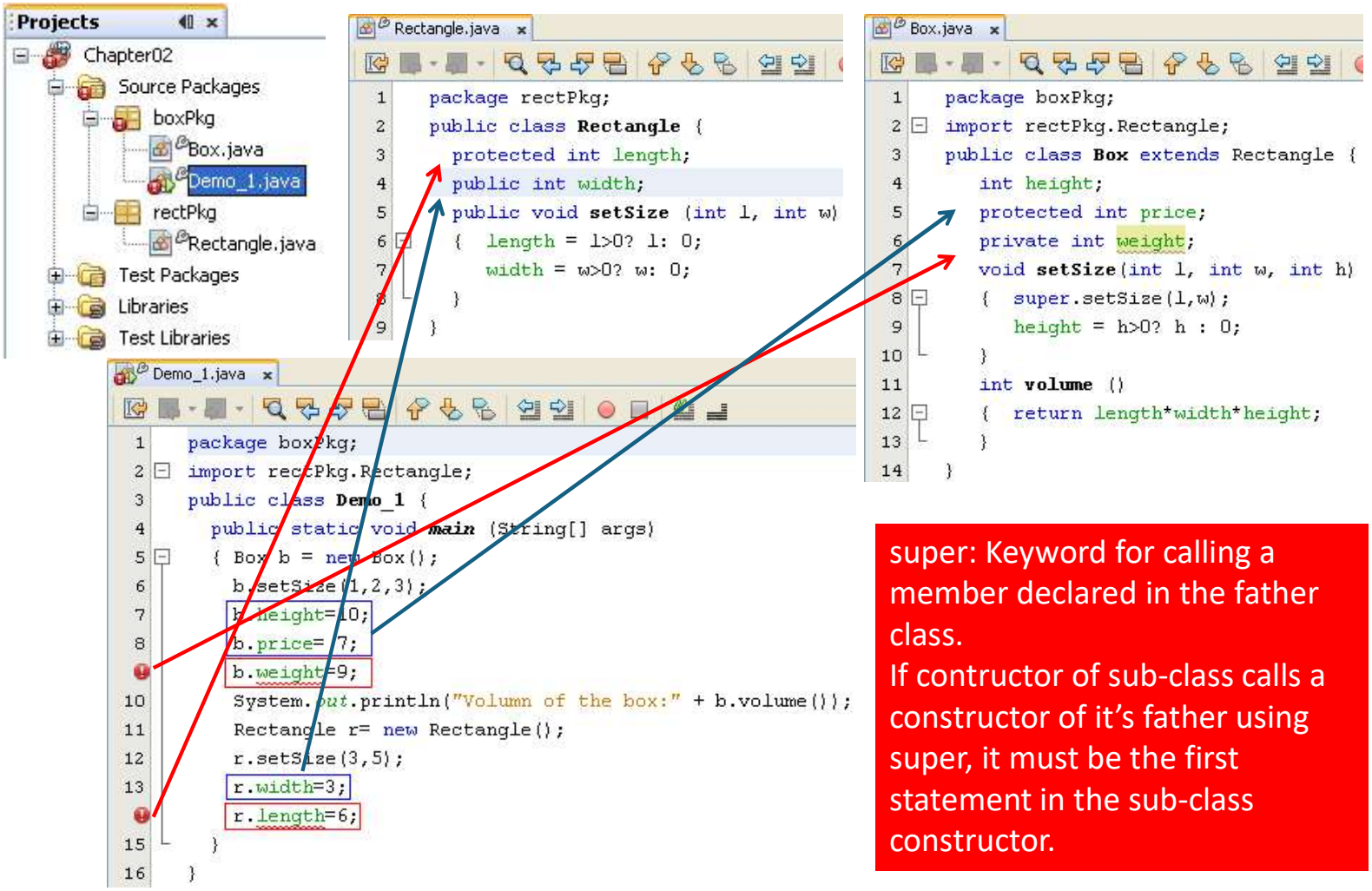
Note: If you don't use any modifier, it is treated as default by default.

Interface is a group of prototyped methods and they will be implemented in a class afterward. **It will be introduced later.**

Access Level

Modifier	Class	Same Package	Subclass-Outside package	World
private	Y	N	N	N
No (default)	Y	Y	N	N
protected	Y	Y	Y	N
public	Y	Y	Y	Y

Quiz?



super: Keyword for calling a member declared in the father class.
If contructor of sub-class calls a constructor of it's father using super, it must be the first statement in the sub-class constructor.

2.4. Overriding methods in sub-classes

Overloading Method

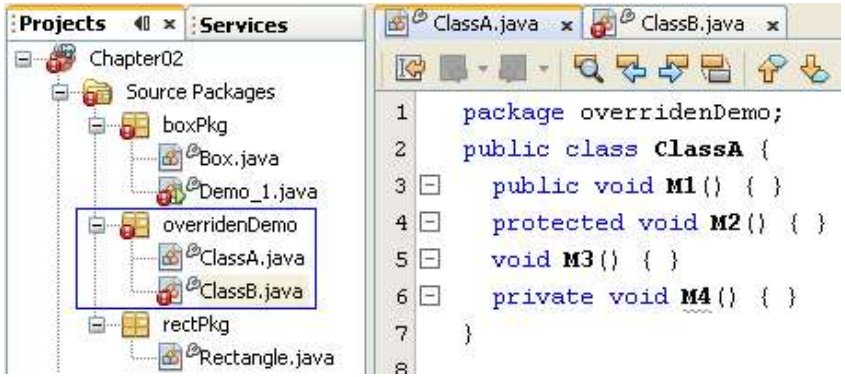
```
/* Overloading methods Demo. */
 public class Box {
   int length=0;
   int width=0;
   int depth=0;
   // Overloading constructors
   public Box(){
   }
   public Box(int l){
       length = l>0? l: 0; // safe state
   }
   public Box(int l, int w){
       length = l>0? l: 0; // safe state
       width = w>0? w: 0;
   }
   public Box(int l, int w, int d){
       length = l>0? l: 0; // safe state
       width = w>0? w: 0;
       depth = d>0? d: 0;
   }
```

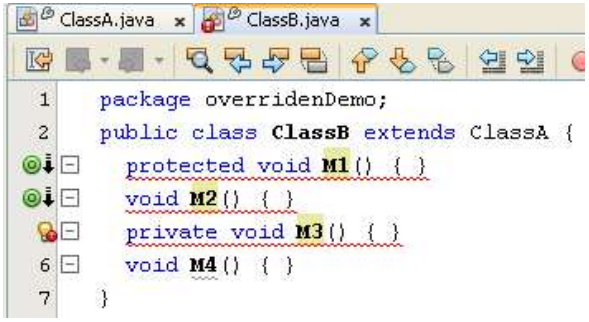
```
   // Overloading methods
   public void setEdge (int l,int w){
       length = l>0? l: 0; // safe state
       width = w>0? w: 0;
   }
   public void setEdge (int l,int w,int d){
       length = l>0? l: 0; // safe state
       width = w>0? w: 0;
       depth = d>0? d: 0;
   }
   public void output(){
     String S= "[" + length + "," + width
               + "," + depth + "]";
     System.out.println(S);
   }
}
```

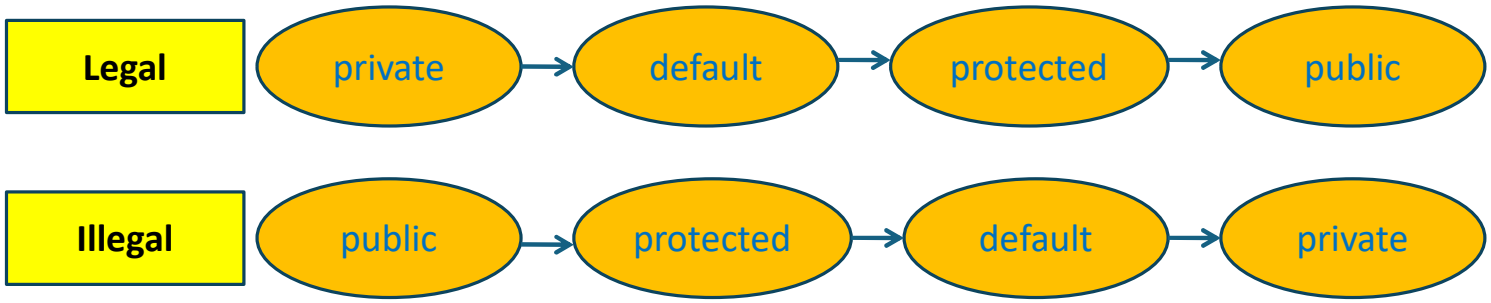
```
Output - FirstPrj (run)
run:
[0,0,0]
[7,3,0]
[90,100,75]
```

```
/* Use the class Box */
 public class BoxUse {
    public static void main(String[] args){
        Box b= new Box();
        b.output();
        b.setEdge(7,3);
        b.output();
        b.setEdge(90,100,75);
        b.output();
    }
}
```

❖ Access Modifier Overridden



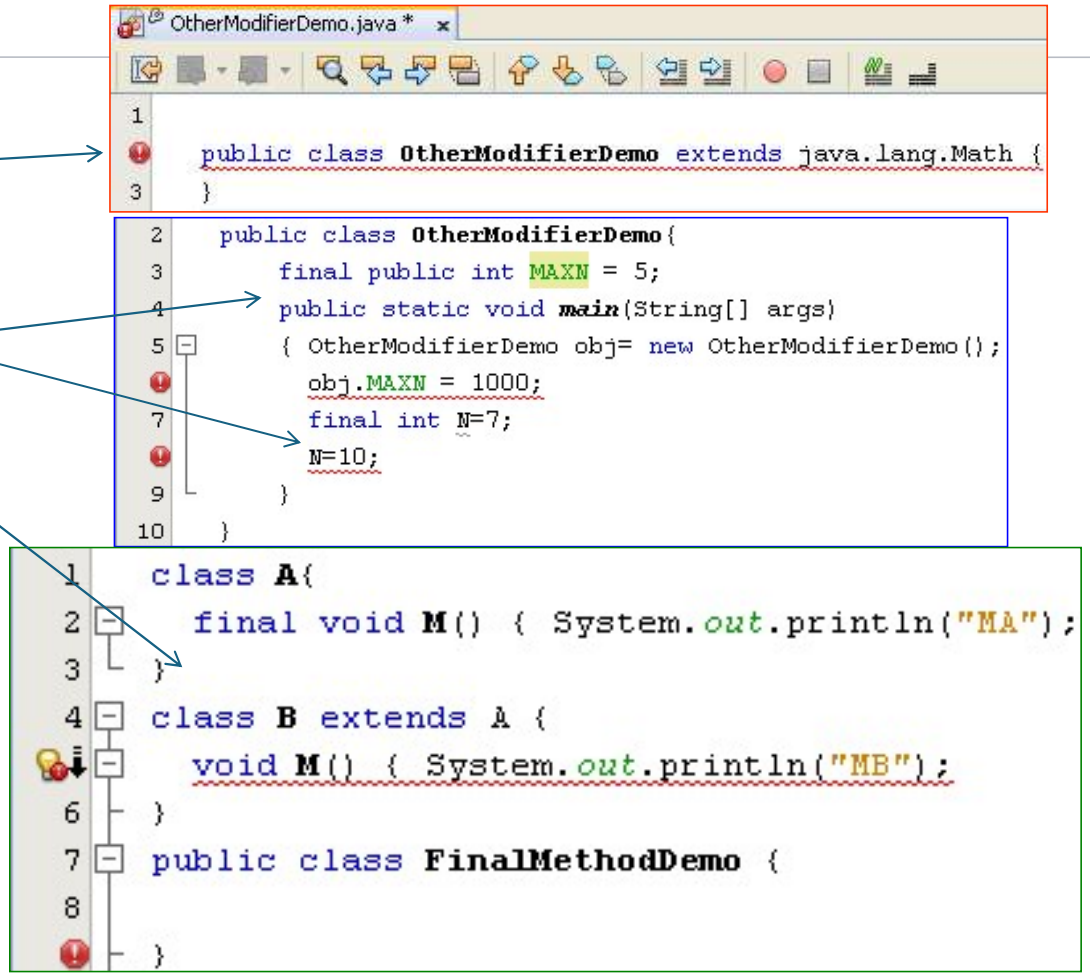




The sub-class must be more opened than it's father

Modifier final

- final class: Class can not have sub-class
- final data is a constant.
- final method: a method can not be overridden.



Modifier static

- Object variable: Variable of each object
- Class Variable: A variable is shared in all objects of class. It is stored separately. It is declared with the modifier static
- Class variable is stored separately. So, it can be accessed as:

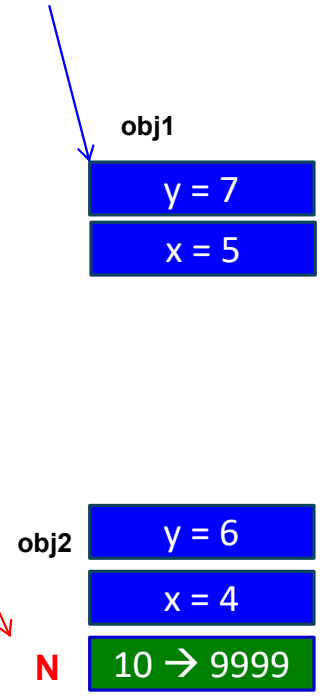
  object.staticVar

  ClassName.staticVar

Modifier *static*: Class variable/ Object variable

```
public class StaticVarDemo {
    static int N=10; // class variable
    int x, y; // object variable
    public StaticVarDemo(int xx, int yy){
        x= xx; y=yy;
    }
    public void setN( int nn){
        N= nn;
    }
    public void output(){
      System.out.println(N + "," + x + "," + y);
    }
}
```

```
public class StaticVarDemoUse {
    public static void main(String args[]){
        StaticVarDemo obj1= new StaticVarDemo(5,7);
        StaticVarDemo obj2= new StaticVarDemo(4,6);
        obj1.output();
        obj2.output();
        obj1.setN(9999);
        obj1.output();
        obj2.output();
        System.out.println(StaticVarDemo.N);
    }
}
```





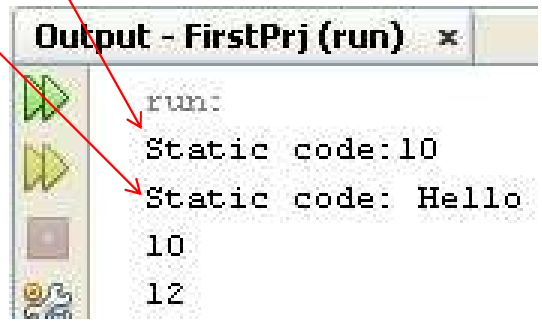
❖ Static code – Free Floating Block

```
public class StaticCodeDemo {
    public static int N=10;
    int x=5, y=7;
    static {
        System.out.println("Static code:" + N);
    }
    int sum(){
        return x+y;
    }
    static {
        System.out.println("Static code: Hello");
    }
}
```

```
public class StaticCodeDemoUse {
    public static void main(String args[]){
        System.out.println(StaticCodeDemo.N);
        StaticCodeDemo obj= new StaticCodeDemo();
        System.out.println(obj.sum());
    }
}
```

The second access

All static code run only one time when the first time the class containing them is accesses



Static method

- It is called as class method/global method and it is called although no object of this class is created.
- Entry point of Java program is a static method
- Syntax for calling it: ClassName.staticMethod(args)
- Static methods:
  - can access class variables and class methods directly only.
  - cannot access instance variables or instance methods directly—they must use an object reference.
  - cannot use the this keyword as there is no instance for this to refer t
- Constants:
  - The static modifier, in combination with the final modifier, is also used to define constants. The final modifier indicates that the value of this field cannot change.

    static final double PI = 3.141592653589793;

Class Constants

❖A class constant is a variable

- whose *scope* is the entire class, and
- whose *value* can never change after it has been initialized.

❖To give it the right scope, simply declare it right inside the class:

```
public class MyClass {
   public static final int myConstant = 4;
}
```

The **final** keyword means its value can't be changed.

2.5. Nested classes

Inner/Outer Class

- **Inner classes** are classes defined within other classes
  - The class that includes the inner class is called the **outer class**
  - There is no particular location where the definition of the inner class (or classes) must be place within the outer class
  - Placing it first or last, however, will guarantee that it is easy to find
- An inner class definition is a member of the outer class in the same way that the instance variables and methods of the outer class are members
  - An inner class is local to the outer class definition
  - The name of an inner class may be reused for something else outside the outer class definition
  - If the inner class is private, then the inner class cannot be accessed by name outside the definition of the outer class

```
public class OuterClass {
    private int x;
    public int getX() {
        return x;
    }
    public void setX(int x) {
        this.x = x;
    }
    public class InnerClass{
        private int y;

        public int getY() {
            return y;
        }
        public void setY(int y) {
            this.y = y;
        }
    }
}
```

```
public class OuterDemo {
    private class InnerDemo{
        public void print() {
            System.out.println("This
                is an inner class");
        }
    }
    void display_Inner() {
        InnerDemo inner = new InnerDemo();
        inner.print();
    }
}
```

```
public class Main {
    public static void main(String[] args) {
         //1
        OuterClass o=new OuterClass();
        o.setX(10);
        OuterClass.InnerClass in=o.new InnerClass();
        in.setY(4);
        System.out.println("Out:"+o.getX()+"\n"+in.getY());
          //2
        OuterDemo outer = new OuterDemo();
        outer.display_Inner();
        }
}
```

2.6. Some useful classes

java.lang.Math

- java.lang
- Math.PI
- Math.abs(-20);
- double c = Math.ceil(7.342); // 8.0
- double f = Math.floor(7.843); // 7.0
- double p = Math.pow(2, 3); // 8.0
- double s = Math.sin(Math.PI/2);//1
- double a = Math.sqrt(9);//3

  .....

java.util.Random

- import java.util.Random;
- Random rd=new Random();
- int a=rd.nextInt(n);//0-(n-1)
- int t = rd.nextInt(max-min+1) +min;//(from min to max)
- Float f=rd.nextFloat();// 0-1
- boolean b=r.nextBoolean();

2.7. Wrapper classes

Wrapper classes

- The wrapper class in Java provides the mechanism to convert primitive into object and object into primitive.

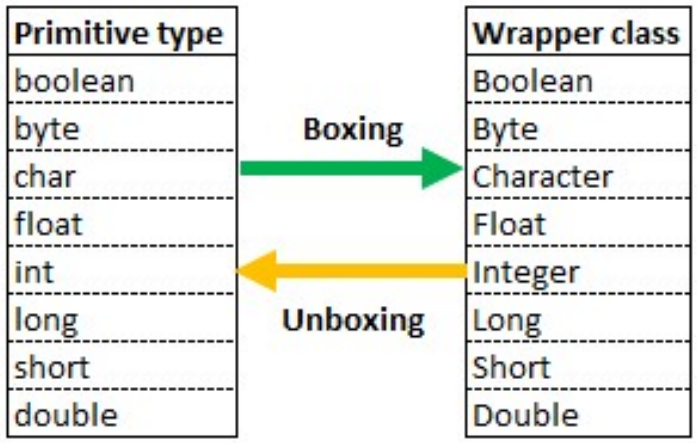
Primitive Type	Size	Minimum Value	Maximum Value	Wrapper Type
char	16-bit	Unicode 0	Unicode 2^{16}-1	Character
byte	8-bit	-128	+127	Byte
short	16-bit	-2^{15} *(-32,768)*	$+2^{15}$-1 *(32,767)*	Short
int	32-bit	-2^{31} *(-2,147,483,648)*	$+2^{31}$-1 *(2,147,483,647)*	Integer
long	64-bit	-2^{63} *(-9,223,372,036,854,775,808)*	$+2^{63}$-1 *(9,223,372,036,854,775,807)*	Long
float	32-bit	32-bit IEEE 754 floating-point numbers		Float
double	64-bit	64-bit IEEE 754 floating-point numbers		Double
boolean	1-bit	`true` or `false`		Boolean
void	-----	-----	-----	Void

Why we need Wrapper Class

- Wrapper Class will convert primitive data types into objects. The objects are necessary if we wish to modify the arguments passed into the method (because primitive types are passed by value).
- The classes in java.util package handles only objects and hence wrapper classes help in this case also.
- Data structures in the Collection framework such as ArrayList and Vector store only the objects (reference types) and not the primitive types.
- The object is needed to support synchronization in multithreading.

Autoboxing and Unboxing

- Since J2SE 5.0, autoboxing and unboxing feature convert primitives into objects and objects into primitives automatically.
- Autoboxing is used to convert primitive data types into corresponding objects.
- Unboxing is used to convert the Wrapper class object into corresponding primitive data types.



❑ Example

```
•Integer y = 567; // //boxing or auto boxing
•Double area(double radius) {
     return Math.PI * radius * radius; //boxing
   }
•Integer wi = 234;
   int times9 = wi * 9; //unboxing
```

- Return primitive types

To return primitive types: using method **type**Value()

```
// make a new wrapper object
Integer i2 = new Integer(42);
// byte
byte b = i2.byteValue();
// short
short s = i2.shortValue();
// double
double d = i2.doubleValue();
```

- to convert from String to a primitive type

  - Using methods (static) of wrapper classes

    ```
    static <type> parseType(String s)
    ```

  - Example

    ```
    String s = "123";
    // int
    int i = Integer.parseInt(s);
    // short
    short j = Short.parseShort(s);
    String txt="13.5";
    Double x =Double.parseDouble(txt);
    ```

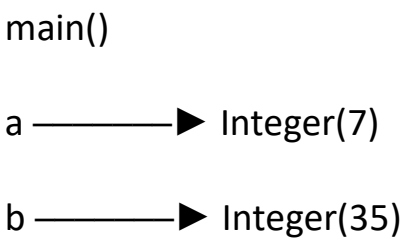
```
public static void main(String[] args) {
    Integer a = 7;
    Integer b = 35;
    System.out.println(a + " " + b);

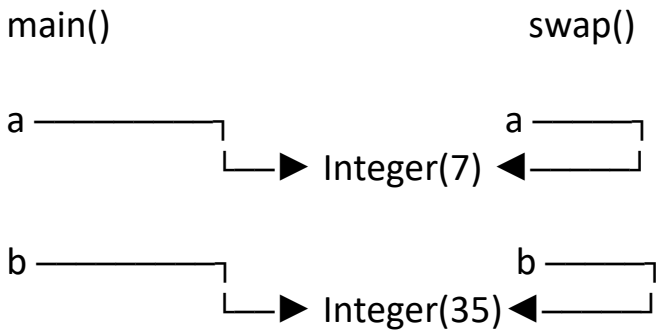
    // swap a with b
    swap(a, b);

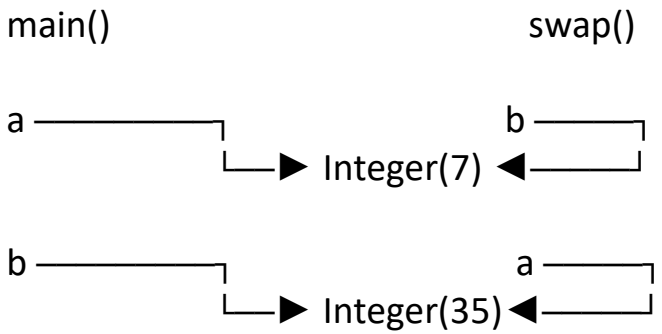
    System.out.println(a + " " + b);
}

public static void swap(Integer a, Integer b) {
    Integer temp = a;
    a = b;
    b = temp;
}
```

What's wrong with this version?


main()
a
Integer(7)
b
Integer(35)


main()
swap()
a
Integer(7)
a
b
Integer(35)
b


main()
swap()
a
Integer(7)
b
b
Integer(35)
a

Java.lang.Integer Class

- static int MAX_VALUE: $2^{31}-1$.
- static int MIN_VALUE : -2^{31}.
- Integer(int value), Integer(String s)
- java.lang.Integer.compare()

```
Integer obj1 = new Integer("25");
Integer obj2 = new Integer("10");
int retval = obj1.compareTo(obj2);
int retval = Integer.compare(obj1,obj2);
```

- byte byteValue(), double doubleValue(), float floatValue()…
- static int parseInt(String s), String toString(), static String toString(int i)
- boolean equals(Object obj)

- Problem with .nextLine() in Java

```
import java.util.Scanner;

Scanner input = new Scanner( System.in );
int option = input.nextInt();
input.nextLine(); // Consume newline left-over
String str1 = input.nextLine();
int option = Integer.parseInt(input.nextLine());
```

Java.lang.Double Class

- Double(double value), Double(String s)
- byte byteValue(), double doubleValue(), float floatValue(), int intValue(), ..
- static int compare(double d1, double d2), int compareTo(Double anotherDouble)
- boolean isNaN(),
- static double parseDouble(String s),
- String toString(), static String toString(double d)

java.lang.Character Class

- static char toUpperCase(char ch)
- static char toLowerCase(char ch)
- static String toString(char c)
- String toString()
- static boolean isWhitespace(char ch)
- static boolean isUpperCase(char ch)
- static boolean isLowerCase(char ch)
- static boolean isLetter(char ch)

Mutable and Immutable objects

- Definition: An ***immutable*** object is an object whose **state cannot be modified** after it is created.

- The String class and all the wrapper classes are *immutable* in Java.
  - As a result, we can't use the wrapper classes to solve the swap problem for primitive types.

- Point objects are *mutable* (ie, not immutable)
  - setLocation and translate change their state
  - We can use this fact to make the swap method work

Review Encapsulation

❖**encapsulation**: Hiding the implementation details of an object from the clients of the object.

❖Encapsulating objects provides *abstraction*, because we can use them without knowing how they work.



- Fields can be declared *private* to indicate that no code outside their own class can change them.
- Declaring a private field, general syntax:

```
private <type> <name>;
```

❑Examples:

```
private int x;
private String name;
```

Access	Keyword	What code has access?
Public	public	All code has access
Private	private	Only code in the same class has access
Protected	protected	Only code in the same class or a *subclass* has access
Package	<nothing>*	Code in the same package has access (this is the default)

- Note: Package-level access is specified by leaving out any keyword. It is ***NOT*** specified using the package keyword, which is used for something else!

- Once fields are private, client code cannot directly access them. The client receives an error such as:

```
PointMain3.java:8: x has private access in Point
System.out.println("p1 is (" + p1.x + ", " + p1.y + ")");
```

How can the client program see what *x* is?

- **accessor**: An instance method that provides information about the state of an object. For example:

```
public int getX() {
    return x;
}
```

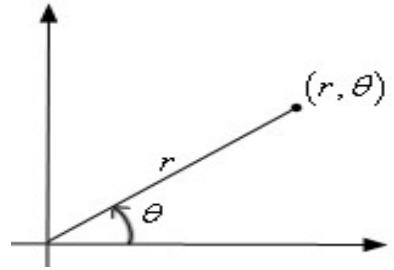
  - This gives clients "read-only" access to the object's fields.

- **mutator**: An instance method that modifies the object's internal state. For example:

```
public void setX(int newX) {
    x = newX;
}
```

Benefits of encapsulation

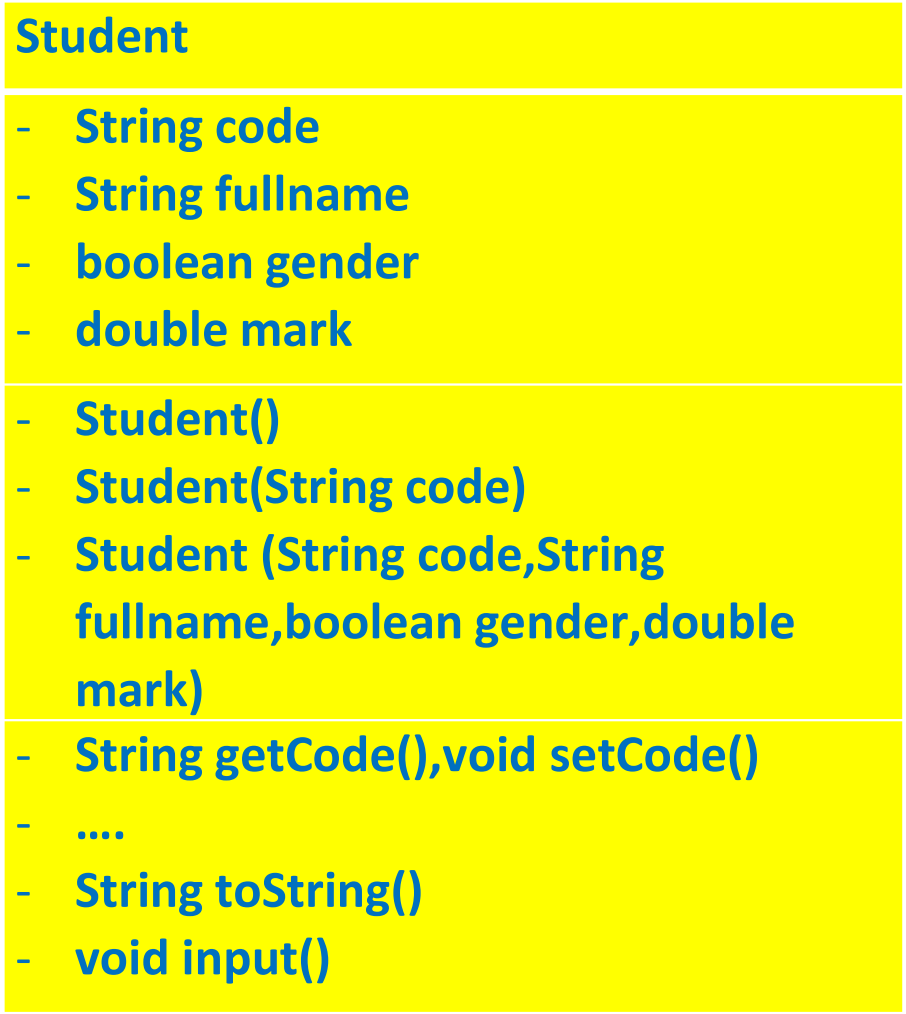
- ❖Encapsulation helps provide a clean layer of abstraction between an object and its clients.
- ❖Encapsulation protects an object from unwanted access by clients.
  - ▪ Would you like any program to be able to modify the BankAccount object with your account information?
- ❖Encapsulation allows the class author to change the internal representation later if necessary.
  - • Example: Changing the Point class to use polar coordinates (a radius r and an angle θ from the origin)



Q&A

❖ Exercise

- Create a new project named “StudentManager”. It contains the file Student.java and Main.java.
- In the file Student.java, you implement the Student class base on the class diagram as below:



Main.java

The program output might look something like:

1. TC = 1 – input a student
2. TC = 2 – Edit a student
3. TC = 3 – toUpper fullname
4. TC = 3 – set mark
5. TC = 4 - toString()
0. TC = 0 - Exit

Enter TC:

Case study

- https://code.ptit.edu.vn/student/question (from 65 to 73, from 150 to 155, from 157 to 160)
- https://codelearn.io/learning/object-oriented-programming-in-java/766304